



POLITECHNIKA MORSKA – WYDZIAŁ INFORMATYKI

PARADYGMATY PROGRAMOWANIA – LAB Programowanie funkcyjne - Haskell część 2
09

Listy: pojęcie, typ i tworzenie

Lista w Haskellu przechowuje dowolną liczbę elementów, ale wszystkie elementy muszą mieć ten sam typ. Typ listy liczb całkowitych zapisujemy jako [Int], a typ listy napisów jako [String].

```
liczby :: [Int]
liczby = [3, 7, 2, 9]
```

```
napisy :: [String]
napisy = ["Haskell", "Prolog", "Lisp"]
```

```
pusta :: [Int]
pusta = []
```

Przykładowe sprawdzenie w interpreterze:

```
> liczby
[3,7,2,9]
> napisy
["Haskell","Prolog","Lisp"]
> pusta
[]
```

Uwaga: Lista [1, "tekst"] jest błędna, ponieważ łączy elementy różnych typów. Do przechowywania wartości różnych typów stosuje się krotki.

Sposoby budowania list

Literały listy

```
oceny :: [Int]
oceny = [5, 4, 3, 5, 2]
```

Wynik: lista zawiera pięć elementów typu Int.

Zakresy

```
> [1..6]
[1,2,3,4,5,6]

> [2,4..12]
[2,4,6,8,10,12]

> [10,9..5]
[10,9,8,7,6,5]
```

Zakresy są wygodne przy tworzeniu danych testowych oraz przy generowaniu sekwencji liczb.

Lista [1..15] jest tym samym co [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]

```
Hugs> [1..15]
[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
Hugs> ['a'..'z']
"abcdefghijklmnopqrstuvwxyz"
Hugs> ['D'..'T']
"DEFGHIJKLMNOPQRST"
```

Tworzenie listy z ustaleniem przyrostu wartości kolejnych elementów

```
Hugs> [2,4..10]
[2,4,6,8,10]
Hugs> [3,6..10]
[3,6,9]
```

cycle tworzy listę nieskończoną bazując na liście wzorcowej – aby uzyskać listę skończoną powinna współpracować z take

```
Hugs> take 11(cycle "XD ")
"XD XD XD XD "
```

repeat tworzy listę nieskończoną poprzez powtarzanie jednej wartości – aby uzyskać listę skończoną powinna współpracować z take

```
Hugs> take 10 (repeat 4)
[4,4,4,4,4,4,4,4,4,4]
```

replicate tworzy listę skończoną poprzez powtarzanie jednej wartości.

```
Hugs> replicate 10 4
[4,4,4,4,4,4,4,4,4,4]
```

Operator dołączania na początek listy: :

```
> 1 : [2, 3, 4]
[1,2,3,4]
```

```
nowaLista :: [Int]
nowaLista = 10 : liczby
-- wynik: [10,3,7,2,9]
```

Operator : dołącza pojedynczy element na początek listy. Lewa strona operatora musi mieć typ elementu listy, a prawa strona musi być listą.

Operator łączenia list: ++

```
> [1, 2] ++ [3, 4, 5]
[1,2,3,4,5]
```

```
> "Para" ++ "dygmaty"
"Paradygmaty"
```

Operator ++ łączy dwie listy tego samego typu. Napis String jest w Haskellu listą znaków, czyli [Char].

List comprehensions

Notacja list comprehensions pozwala budować listy na podstawie innych list, warunków i transformacji.

```
kwadratyParzystych :: [Int]
kwadratyParzystych = [x * x | x <- [1..10], even x]
-- wynik: [4,16,36,64,100]
```

```
pary :: [(Int, Int)]
pary = [(x, y) | x <- [1..3], y <- [1..2]]
-- wynik: [(1,1),(1,2),(2,1),(2,2),(3,1),(3,2)]
```

Podstawowe funkcje działające na listach

Poniższe funkcje pochodzą z modułu Prelude, dlatego są dostępne bez dodatkowego importu. Przy funkcjach częściowych, takich jak head lub tail, należy zachować ostrożność przy pustej liście.

Funkcja	Znaczenie	Przykład i wynik
head	zwraca pierwszy element listy	head [8, 3, 5] -> 8
tail	zwraca listę bez pierwszego elementu	tail [8, 3, 5] -> [3,5]
last	zwraca ostatni element listy	last [8, 3, 5] -> 5
init	zwraca listę bez ostatniego elementu	init [8, 3, 5] -> [8,3]
null	sprawdza, czy lista jest pusta	null [] -> True
length	zwraca liczbę elementów	length [8, 3, 5] -> 3
reverse	odwraca kolejność elementów	reverse [8, 3, 5] -> [5,3,8]
take	pobiera n pierwszych elementów	take 2 [8, 3, 5] -> [8,3]
drop	pomija n pierwszych elementów	drop 2 [8, 3, 5] -> [5]
!!	pobiera element o indeksie liczonym od 0	[8, 3, 5] !! 1 -> 3
elem	sprawdza, czy element występuje na liście	3 `elem` [8, 3, 5] -> True

Uwaga: Wywołania head [], tail [], last [] oraz indeksowanie poza zakresem powodują błąd wykonania.

Funkcje agregujące listy liczbowe i porównywalne

Funkcja	Znaczenie	Przykład i wynik
sum	suma elementów	sum [2, 4, 6] -> 12
product	iloczyn elementów	product [2, 4, 6] -> 48
maximum	największy element	maximum [2, 4, 6] -> 6
minimum	najmniejszy element	minimum [2, 4, 6] -> 2

Przykład funkcji obliczającej średnią z listy liczb:

```
srednia :: [Double] -> Double
srednia xs = sum xs / fromIntegral (length xs)
```

```
> srednia [3.0, 4.0, 5.0]
4.0
```

Funkcje wyższego rzędu na listach

Funkcje wyższego rzędu przyjmują inne funkcje jako argumenty albo zwracają funkcje jako wynik. Są one typowym narzędziem programowania funkcyjnego.

map

Funkcja map stosuje podaną funkcję do każdego elementu listy.

```
podwojone :: [Int]
podwojone = map (*2) [1, 2, 3, 4]
-- wynik: [2,4,6,8]
```

```
pierwszeLitery :: [Char]
pierwszeLitery = map head ["Ada", "Edsger", "Grace"]
-- wynik: "AEG"
```

filter

Funkcja filter pozostawia tylko te elementy, które spełniają warunek logiczny.

```
wiekszeOdZera :: [Int]
wiekszeOdZera = filter (>0) [-3, 0, 4, 7, -1]
-- wynik: [4,7]
```

```
długieNapisy :: [String]
długieNapisy = filter (\s -> length s >= 6) ["C", "Python", "Haskell"]
-- wynik: ["Python","Haskell"]
```

foldl i foldr

Funkcje foldl i foldr redukują listę do jednej wartości. Różnią się kierunkiem przetwarzania: foldl składa od lewej, a foldr od prawej.

```
sumaFold :: [Int] -> Int
sumaFold xs = foldl (+) 0 xs
```

```
iloczynFold :: [Int] -> Int
iloczynFold xs = foldr (*) 1 xs
```

```
> sumaFold [1, 2, 3, 4]
10
> iloczynFold [2, 3, 4]
24
```

all i any

```
> all even [2, 4, 6]
True
```

```
> any (<0) [4, 2, -1, 8]
True
```

all sprawdza, czy wszystkie elementy spełniają warunek, natomiast any sprawdza, czy warunek spełnia co najmniej jeden element.

zip, unzip i zipWith

Funkcja zip łączy dwie listy w listę par. Funkcja unzip wykonuje operację odwrotną. zipWith łączy elementy dwóch list za pomocą wskazanej funkcji.

```
> zip ["Ala", "Jan", "Ola"] [5, 4, 3]
[("Ala",5),("Jan",4),("Ola",3)]
```

```
> unzip [("Ala",5),("Jan",4)]
[("Ala","Jan"),[5,4]]
```

```
> zipWith (+) [1, 2, 3] [10, 20, 30]
[11,22,33]
```

Własne funkcje na listach: dopasowanie wzorców

Dopasowanie wzorców pozwala rozróżnić listę pustą, listę jednoelementową oraz listę z głową i ogonem. Jest to jeden z najważniejszych idiomów Haskella.

```
sumaRekurencyjna :: [Int] -> Int
sumaRekurencyjna [] = 0
sumaRekurencyjna (x:xs) = x + sumaRekurencyjna xs
```

```
liczDodatnie :: [Int] -> Int
liczDodatnie [] = 0
liczDodatnie (x:xs)
  | x > 0      = 1 + liczDodatnie xs
  | otherwise = liczDodatnie xs
```

```
bezpiecznyPierwszy :: [a] -> Maybe a
bezpiecznyPierwszy [] = Nothing
bezpiecznyPierwszy (x:_) = Just x
```

Przykłady wywołania:

```
> sumaRekurencyjna [4, 1, 5]
10
> liczDodatnie [-2, 0, 3, 8]
2
> bezpiecznyPierwszy []
Nothing
> bezpiecznyPierwszy ["Haskell", "ML"]
Just "Haskell"
```

Krotki: pojęcie, typ i zastosowanie

Krotka grupuje ustaloną liczbę wartości, które mogą mieć różne typy. W przeciwieństwie do listy długość krotki jest częścią jej typu.

```
student :: (String, Int)
student = ("Anna", 21)
```

```
pomiar :: (String, Double, Bool)
pomiar = ("CPU", 72.5, True)
```

Typ (String, Int) oznacza parę: napis i liczba całkowita. Typ (String, Double, Bool) oznacza trójkę: napis, liczba rzeczywista i wartość logiczna.

Funkcje działające na krotkach

Funkcja / technika	Znaczenie	Przykład i wynik
fst	zwraca pierwszy element pary	fst ("Ala", 5) -> "Ala"
snd	zwraca drugi element pary	snd ("Ala", 5) -> 5
dopasowanie wzorca	wydobywa wiele pól i działa także dla trójek	opisStudenta (imie, wiek) = imie ++ ": " ++ show wiek
zip	tworzy listę par z dwóch list	zip ["a", "b"] [1,2] -> [("a",1), ("b",2)]
unzip	dzieli listę par na dwie listy	unzip [("a",1), ("b",2)] -> (["a", "b"], [1,2])

fst i snd

```
osoba :: (String, Int)
osoba = ("Tomasz", 22)
```

```
> fst osoba
"Tomasz"
> snd osoba
22
```

Uwaga: *fst i snd działają tylko dla par. Dla trójek i większych krotek stosuje się dopasowanie wzorców.*

Dopasowanie wzorców w funkcjach na krotkach

```
opisOsoby :: (String, Int) -> String
opisOsoby (imie, wiek) = imie ++ ", wiek: " ++ show wiek
```

```
czyAktywnyPomiar :: (String, Double, Bool) -> Bool
czyAktywnyPomiar (_, _, aktywny) = aktywny
```

```
> opisOsoby ("Maria", 20)
"Maria, wiek: 20"
> czyAktywnyPomiar ("RAM", 63.4, True)
True
```

Funkcje zwracające krotki

```
statystyka :: [Int] -> (Int, Int, Int)
statystyka xs = (minimum xs, maximum xs, length xs)
```

```
> statystyka [7, 2, 9, 4]
(2,9,4)
```

Funkcja statystyka zwraca jednocześnie trzy wartości: minimum, maksimum i liczbę elementów.

Listy krotek: praktyczny model danych

Częstym rozwiązaniem jest przechowywanie wielu rekordów jako listy krotek. Przykład: lista studentów z liczbą punktów.

```
type Student = (String, Int)
```

```
studenci :: [Student]
studenci = [("Anna", 82), ("Jan", 55), ("Ola", 91), ("Piotr", 64)]
```

```
punkty :: Student -> Int
punkty (_, p) = p
```

```
zaliczeni :: [Student] -> [Student]
zaliczeni xs = filter (\s -> punkty s >= 60) xs
```

```
nazwyStudentow :: [Student] -> [String]
nazwyStudentow xs = map fst xs
```

```
> zaliczeni studenci
[("Anna",82),("Ola",91),("Piotr",64)]
> nazwyStudentow studenci
["Anna","Jan","Ola","Piotr"]
```

W tym przykładzie typ Student jest aliasem dla pary (String, Int). Alias nie tworzy nowego typu, ale poprawia czytelność kodu.